

Ejercicio 5

1. Indique si el diseño corresponde a un framework o a una librería. Justifique adecuadamente.

Es un **framework** ya que es un código ya completo que te llama a vos cuando te necesita (en puntos definidos)

Nosotros llamamos a `emit()` y `registerListener()`, pero cuando se emite el dispatcher toma el control e invoca a `onEvent()` del `Listener` (basicamente el framework decide cuando y a quien llamar)

Es un framework de caja negra, se extiende por composición

2. Identifique si existe Inversión de Control. En caso afirmativo, indique dónde ocurre. Justifique claramente.

Si hay inversión de control, en `dispatch()`, pasa en la línea `bl.onEvent(event)`, vos definís que hace el `onEvent()`, pero el framework decide cuando llamarlo

`setDispatcher()` es una inyección de dependencias ya que no construís el dispatcher adentro sino que lo metes desde afuera

3. Cree un mecanismo para registrar en consola cada evento despachado, mostrando: nombre del evento, fecha y hora. Restricción: no se permite modificar el código provisto.

No hace falta modificar código, ya existe el hotspot `BeaconListener`, se instancia y se registra

```
1 public class LoggingListener implements BeaconListener {
2     @Override
3     public void onEvent(BeaconEvent event){
4         System.out.println(event.getTimestamp() + " " + event.getName());
5     }
6 }
```

```
1 beacons.registerListener(new LoggingListener());
```

4. Se desea que los Listeners reciban únicamente aquellos eventos que sean de su interés, evitando que reaccionen a eventos irrelevantes.

- Proponga una solución de diseño
- Explique cómo un Listener determina qué eventos le interesan.

- ¿Considera que la solución es una extensión o una instanciación? Justifique.

Defino una clase que sea un listener selectivo

```
1  public abstract class SelectiveListener implements BeaconListener {
2      @Override
3      public final void onEvent(BeaconEvent event){
4          if (meInteresa(event))
5              handle(event);
6      }
7
8      protected abstract boolean meInteresa(BeaconEvent event);
9      protected abstract void handle(BeaconEvent event);
10 }
```

Esta clase define un patron para que las instanciaciones de esta clase digan que les interesa y que hacer cuando algo les interesa. Pueden retornar true si el evento tiene un nombre que les interesa o algo asi. Para manejarlo tambien depende de cada implementacion

Es una **instanciacion** por composicion ya que estamos usando los hotspots dd el framework sin tocar `StreamBeacons`, la decision de hacer una clase abstracta es nuestra y no quita el hecho de que estamos enchufados a `BeaconListener`

5. Indicar Verdadero o Falso y justificar en cada caso:

a. El método dispatch de BeaconDispatcher es un Template Method.

Es mentira ya que nada mas esta definiendo que metodo tienen que cumplir todos los listeners, no hay una serie de pasos definida, es simplemente una declaracion abstracta

b. El método register es un método hook.

Es mentira porque un hook se refiere a un metodo con comportamiento por defecto (casi siempre vacio) que la subclase puede sobrescribir que esta diseñado para que las clases lo puedan sobrescribir o extender de forma opcional. Aca si o si tiene que implementar un metodo register el cual no es parte de nada

c. El método onEvent constituye un hotspot.

Si constituye un hotspot ya que define que va a hacer cada aplicacion cuando le llega un evento

d. La clase StreamBeacons no puede considerarse un frozenspot porque permite cambiar el BeaconDispatcher mediante el método setDispatcher.

Es mentira porque la clase en si misma es final y la manera que tiene de hacer las cosas nunca va a cambiar, su manera de variarla es cambiarle los dispatcher. Es mas agregarle una configuracion que hacerlo un hotspot, en definitiva es un frozenspot configurable con `setDispatcher()`

e. El diseño no permite extender el comportamiento de BeaconDispatcher, por lo tanto, es de caja blanca.

Es falso ya que la reutilizacion por herencia es de caja blanca, lo que es de caja negra es la composicion como en este caso.

f. El uso de interfaces confirma que se trata de un diseño de caja negra.

Es mentira ya que una interfaz no asegura al 100% que es de caja negra, lo que lo hace es la composicion que no depende de como este hecho por adentro. Se pueden tener interfaces y que sea caja blanca

g. Dado que DoNotDisturbDispatcher no hereda del DefaultDispatcher, se puede afirmar que es un diseño de caja negra.

Lo mismo que antes, que la extension no sea por herencia no nos asegura nada, ya que `StreamBeacons` lo va a seguir usando a traves de la misma interfaz `BeaconDispatcher`, la herencia puede ser una herramienta para no repetir codigo y solo cambiar alguna funcionalidad